

PYSPARK · JSON TRANSFORMATIONS

JSON Transformations in Spark

84 tasks and solutions — from basic `select()` to nested arrays, `from_json()/to_json()`, windows, and the Kafka streaming pipeline.

Big Data / Spark Structured Streaming interview prep

Roadmap

25 levels · 84 tasks

- | | | | | | | | | |
|----|---------------------------------|--------|----|------------------------|--------|----|-----------------------------|--------|
| 01 | Basic DataFrame Transformations | T1-5 | 10 | Duplicate Handling | T41-42 | 19 | selectExpr() | T68-69 |
| 02 | withColumn() | T6-9 | 11 | Array Transformations | T43-48 | 20 | alias() | T70 |
| 03 | Filtering | T10-14 | 12 | Nested JSON Objects | T49-51 | 21 | Partition Transformations | T71-74 |
| 04 | Sorting | T15-17 | 13 | Nested Arrays | T52-55 | 22 | Set Operations | T75-77 |
| 05 | String Transformations | T18-24 | 14 | Explode Orders | T56-57 | 23 | Window Transformations | T78-81 |
| 06 | Date Transformations | T25-28 | 15 | Calculate Order Values | T58-60 | 24 | Conditional Transformations | T82-83 |
| 07 | Aggregations | T29-36 | 16 | Array Functions | T61-63 | 25 | Final Mini Project | T84 |
| 08 | groupBy() + filter() | T37 | 17 | JSON Functions | T64-65 | | | |
| 09 | Null Handling | T38-40 | 18 | from_json() | T66-67 | | | |

Display the data

```
df.show()
```

Display the schema

```
df.printSchema()
```

Select specific columns

Select: user_id, name, age, city, salary

```
df.select(  
    "user_id",  
    "name",  
    "age",  
    "city",  
    "salary"  
) .show()
```

Select nested JSON fields

Select: name, area, postal code

```
df.select(  
    "name",  
    "address.area",  
    "address.postal_code"  
) .show()
```

Rename columns

name -> full_name, salary -> monthly_salary

```
df.withColumnRenamed("name", "full_name") \
  .withColumnRenamed("salary", "monthly_salary") \
  .show()
```

Create a yearly salary column

```
df.withColumn(  
    "yearly_salary",  
    col("salary") * 12  
) .show()
```


Create an age category

age >= 30 -> Senior, otherwise -> Junior

```
df.withColumn(  
    "age_category",  
    when(col("age") >= 30, "Senior")  
    .otherwise("Junior")  
) .show()
```

Create salary category

```
df.withColumn(  
    "salary_category",  
    when(col("salary") >= 15000, "High")  
    .when(col("salary") >= 10000, "Medium")  
    .otherwise("Low")  
) .show()
```

Add a constant column

```
df.withColumn(  
    "source",  
    lit("users_json")  
) .show()
```

Find active users

```
df.filter(  
    col("is_active") == True  
) .show()
```

Find users older than 30

```
df.filter(  
    col("age") > 30  
) .show()
```

Find users from Cairo

```
df.filter(  
    col("city") == "Cairo"  
) .show()
```

Find users from Cairo or Giza

```
df.filter(  
    col("city").isin("Cairo", "Giza")  
) .show()
```

Multiple conditions

Active users from Cairo earning more than 10,000

```
df.filter(  
    (col("is_active") == True) &  
    (col("city") == "Cairo") &  
    (col("salary") > 10000)  
) .show()
```


Sort by salary ascending

```
df.orderBy("salary").show()
```

Sort by salary descending

```
df.orderBy(  
    col("salary").desc()  
) .show()
```

Sort by department and salary

```
df.orderBy(  
    col("department"),  
    col("salary").desc()  
) .show()
```

Convert names to uppercase

```
df.withColumn(  
    "name_upper",  
    upper("name")  
) .show()
```

Convert names to lowercase

```
df.withColumn(  
    "name_lower",  
    lower("name")  
) .show()
```

Extract first name

```
df.withColumn(  
    "first_name",  
    split(col("name"), " ").getItem(0)  
) .show()
```

Extract last name

```
df.withColumn(  
    "last_name",  
    split(col("name"), " ").getItem(1)  
) .show()
```

Calculate name length

```
df.withColumn(  
    "name_length",  
    length("name")  
) .show()
```


Replace part of an email

```
df.withColumn(  
    "email",  
    regexp_replace("email", "@example.com", "@company.com")  
) .show()
```

Create username from email

```
df.withColumn(  
    "username",  
    split("email", "@").getItem(0)  
) .show()
```

Convert join_date to date

```
df.withColumn(  
    "join_date",  
    to_date("join_date")  
) .show()
```

Extract joining year

```
df.withColumn(  
    "join_year",  
    year("join_date")  
) .show()
```

Extract joining month

```
df.withColumn(  
    "join_month",  
    month("join_date")  
) .show()
```

Calculate years since joining

```
df.withColumn(  
    "years_since_joining",  
    year(current_date()) - year("join_date")  
) .show()
```

Count users

```
df.count()
```

Average salary

```
df.select(  
    avg("salary").alias("average_salary")  
) .show()
```


Maximum salary

```
df.select(  
    max("salary").alias("max_salary")  
) .show()
```

Minimum salary

```
df.select(  
    min("salary").alias("min_salary")  
) .show()
```

Total salaries

```
df.select(  
    sum("salary").alias("total_salary")  
) .show()
```

Count users per city

```
df.groupby("city") \
    .count() \
    .show()
```

Average salary per department

```
df.groupBy("department") \
    .agg(
        avg("salary").alias("avg_salary")
    ) \
    .show()
```

Multiple aggregations

```
df.groupBy("department").agg(  
    count("*").alias("employees"),  
    avg("salary").alias("avg_salary"),  
    min("salary").alias("min_salary"),  
    max("salary").alias("max_salary"),  
    sum("salary").alias("total_salary")  
) .show()
```

Departments with avg salary > 10,000

```
df.groupby("department") \
    .agg(
        avg("salary").alias("avg_salary")
    ) \
    .filter(
        col("avg_salary") > 10000
    ) \
    .show()
```

Count nulls

```
df.select([
    count(
        when(col(c).isNull(), c)
    ).alias(c)
    for c in df.columns
]).show()
```


Drop rows containing nulls

```
df.dropna().show()
```

Fill null values

```
df.fillna({  
    "salary": 0,  
    "city": "Unknown"  
}).show()
```

Remove duplicate users

```
df.dropDuplicates().show()
```

Remove duplicates based on user_id

```
df.dropDuplicates(["user_id"]).show()
```

Display skills

skills = ["Python", "SQL", "Spark"]

```
df.select(  
    "name",  
    "skills"  
) .show(truncate=False)
```

Get first skill

```
df.select(  
    "name",  
    col("skills")[0].alias("first_skill")  
) .show()
```

Get number of skills

```
df.select(  
    "name",  
    size("skills").alias("number_of_skills")  
) .show()
```

Check whether user knows Python

```
df.select(  
    "name",  
    array_contains("skills", "Python").alias("knows_python")  
) .show()
```


Explode skills

One row per (name, skill) pair

```
df.select(  
    "name",  
    explode("skills").alias("skill")  
) .show()
```

explode_outer()

Like explode(), but keeps rows with empty/null arrays

```
df.select(  
    "name",  
    explode_outer("skills").alias("skill")  
) .show()
```

Select the entire address

address: street, area, postal_code

```
df.select(  
    "name",  
    "address"  
) .show(truncate=False)
```

Select individual nested fields

```
df.select(  
    "name",  
    col("address.street"),  
    col("address.area"),  
    col("address.postal_code")  
) .show()
```

Create a full address

```
df.withColumn(  
    "full_address",  
    concat_ws(  
        ", ",  
        "address.street",  
        "address.area",  
        "address.postal_code"  
    )  
) .show(truncate=False)
```

Display orders

orders: [{order_id, product, quantity, price}, ...]

```
df.select(  
    "name",  
    "orders"  
) .show(truncate=False)
```

Get first order

```
df.select(  
    "name",  
    col("orders")[0].alias("first_order")  
) .show(truncate=False)
```

Get first order's product

```
df.select(  
    "name",  
    col("orders")[0]["product"].alias("product")  
) .show()
```


Count orders

```
df.select(  
    "name",  
    size("orders").alias("order_count")  
) .show()
```

Explode orders

```
orders_df = df.select(  
    "user_id",  
    "name",  
    explode("orders").alias("order")  
)  
orders_df.show(truncate=False)
```

Extract order fields

order: order_id, product, quantity, price

```
orders_df.select(  
    "user_id",  
    "name",  
    col("order.order_id").alias("order_id"),  
    col("order.product").alias("product"),  
    col("order.quantity").alias("quantity"),  
    col("order.price").alias("price")  
) .show()
```

Calculate total order value

```
orders_df = orders_df.withColumn(  
    "total",  
    col("order.quantity") * col("order.price")  
)  
orders_df.show()
```

Find expensive orders

```
orders_df.filter(  
    col("total") > 10000  
) .show()
```

Calculate total spending per user

```
orders_df.groupBy(  
    "user_id",  
    "name"  
) .agg(  
    sum(  
        col("order.quantity") * col("order.price")  
    ) .alias("total_spending")  
) .show()
```

Sort skills

```
df.select(  
    "name",  
    sort_array("skills").alias("sorted_skills")  
) .show(truncate=False)
```

Reverse skills

```
df.select(  
    "name",  
    reverse("skills").alias("reversed_skills")  
) .show(truncate=False)
```


Join skills into a string

```
df.select(  
    "name",  
    concat_ws(",", "skills").alias("skills_string")  
) .show()
```

Convert a struct to JSON

Result: {"street": "Nile Street", "area": "Maadi", "postal_code": "11728"}

```
df.select(  
    "name",  
    to_json("address").alias("address_json")  
) .show(truncate=False)
```

Convert orders to JSON

```
df.select(  
    "name",  
    to_json("orders").alias("orders_json")  
) .show(truncate=False)
```

Parse JSON

Build json_df of raw strings + a StructType schema first

```
json_df = spark.createDataFrame([
    ('{"id":101,"name":"Ahmed","age":25}',),
    ('{"id":102,"name":"Youssef","age":28}',),
], ["json_string"])

schema = StructType([
    StructField("id", IntegerType()),
    StructField("name", StringType()),
    StructField("age", IntegerType())
])

parsed_df = json_df.withColumn(
    "data",
    from_json("json_string", schema)
)
parsed_df.show(truncate=False)
```

Extract parsed fields

Pattern used with Kafka: value -> cast(string) -> from_json() -> select("data.")*

```
parsed_df.select(  
  "data.id",  
  "data.name",  
  "data.age"  
) .show()
```

Use SQL expressions instead of select()

```
df.selectExpr(  
    "name",  
    "age",  
    "salary",  
    "salary * 12 AS yearly_salary"  
) .show()
```

Use SQL expressions for filtering

```
df.selectExpr(  
    "name",  
    "department",  
    "salary"  
) .filter(  
    "salary > 10000"  
) .show()
```

Rename expressions using alias()

```
df.select(  
    col("name").alias("employee_name"),  
    col("salary").alias("monthly_salary"),  
    (col("salary") * 12).alias("yearly_salary")  
) .show()
```


Check number of partitions

```
df.rdd.getNumPartitions()
```

Repartition

```
df2 = df.repartition(4)
```

Repartition by city

```
df2 = df.repartition(  
    4,  
    "city"  
)
```

Coalesce

repartition(): can increase/decrease, usually shuffles. coalesce(): usually decreases, avoids full shuffle.

```
df3 = df.coalesce(2)
```

Union

```
df1 = df.filter(age > 30) df2 = df.filter(salary > 15000)
```

```
df1.union(df2).show()
```

Union by column name

```
df1.unionByName(df2).show()
```

Distinct

```
df1.union(df2).distinct().show()
```

Rank employees by salary

```
from pyspark.sql.window import Window

window_spec = Window.orderBy(
    col("salary").desc()
)

df.withColumn(
    "rank",
    rank().over(window_spec)
).show()
```


Rank employees inside each department

```
window_spec = Window.partitionBy(
    "department"
).orderBy(
    col("salary").desc()
)

df.withColumn(
    "rank",
    rank().over(window_spec)
).show()
```

row_number()

```
df.withColumn(  
    "row_number",  
    row_number().over(window_spec)  
) .show()
```

dense_rank()

```
df.withColumn(  
    "dense_rank",  
    dense_rank().over(window_spec)  
) .show()
```

Categorize users

```
df.withColumn(  
    "level",  
    when(col("salary") >= 15000, "Senior")  
    .when(col("salary") >= 10000, "Mid-Level")  
    .otherwise("Junior")  
) .show()
```

Multiple conditions

```
df.withColumn(  
  "employee_type",  
  when(  
    (col("age") >= 30) &  
    (col("salary") >= 12000),  
    "Experienced"  
  )  
  .when(  
    col("age") < 25,  
    "Young"  
  )  
  .otherwise("Regular")  
) .show()
```

Build the final employee dataset (part 1 of 2)

Target columns: *user_id, first_name, last_name, city, department, age, salary, yearly_salary, salary_category, join_year, number_of_skills, number_of_orders, full_address*

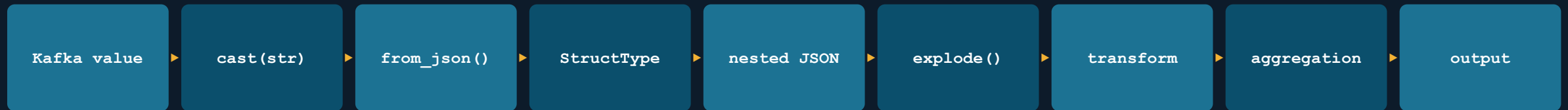
```
final_df = (  
    df  
    .withColumn("first_name", split("name", " ").getItem(0))  
    .withColumn("last_name", split("name", " ").getItem(1))  
    .withColumn("yearly_salary", col("salary") * 12)  
    .withColumn(  
        "salary_category",  
        when(col("salary") >= 15000, "High")  
        .when(col("salary") >= 10000, "Medium")  
        .otherwise("Low")  
    )  
    .withColumn("join_year", year("join_date"))  
    .withColumn("number_of_skills", size("skills"))  
    # ... continued on next slide -->
```

Build the final employee dataset (part 2 of 2)

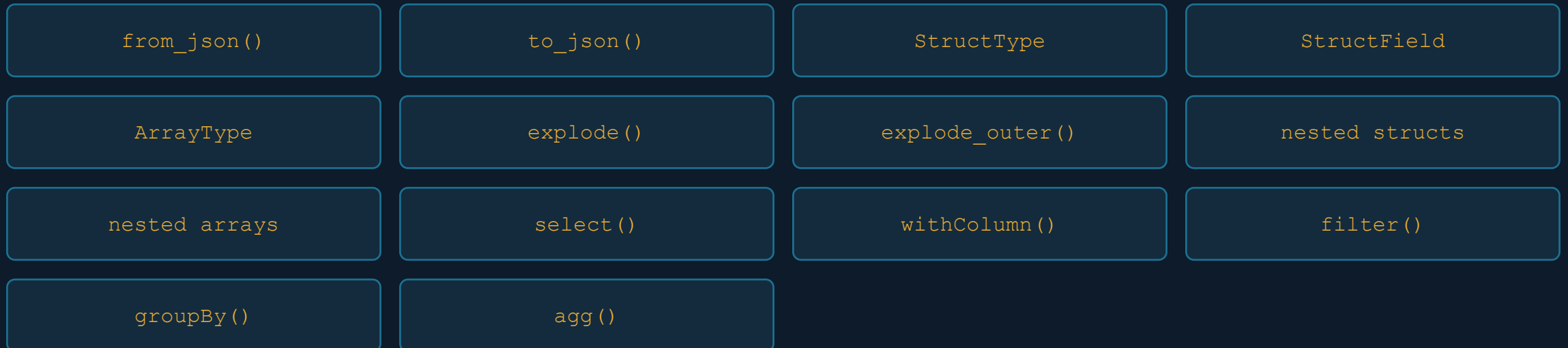
```
final_df = (  
    df  
    # ... (first_name ... number_of_skills, see previous slide)  
    .withColumn("number_of_orders", size("orders"))  
    .withColumn(  
        "full_address",  
        concat_ws(  
            ", ",  
            "address.street",  
            "address.area",  
            "address.postal_code"  
        )  
    )  
    .select(  
        "user_id", "first_name", "last_name", "city",  
        "department", "age", "salary", "yearly_salary",  
        "salary_category", "join_year", "number_of_skills",  
        "number_of_orders", "full_address"  
    )  
)  
  
final_df.show(truncate=False)
```

From JSON Practice to Kafka Streaming

This is the bridge between the JSON tasks above and Spark Structured Streaming + Kafka projects.



Pay extra attention to these for your streaming goal:



What This Dataset Lets You Practice

Category -> functions / transformations (1 of 2)

CATEGORY	FUNCTIONS / TRANSFORMATIONS
Selection	<code>select()</code> , <code>selectExpr()</code>
Columns	<code>withColumn()</code> , <code>withColumnRenamed()</code> , <code>alias()</code>
Filtering	<code>filter()</code> , <code>where()</code>
Sorting	<code>orderBy()</code> , <code>sort()</code>
Conditions	<code>when()</code> , <code>otherwise()</code>
Constants	<code>lit()</code>
Strings	<code>upper()</code> , <code>lower()</code> , <code>length()</code> , <code>split()</code> , <code>regexp_replace()</code>
Dates	<code>to_date()</code> , <code>year()</code> , <code>month()</code> , <code>current_date()</code>
Aggregation	<code>count()</code> , <code>sum()</code> , <code>avg()</code> , <code>min()</code> , <code>max()</code>
Grouping	<code>groupBy()</code> , <code>agg()</code>

What This Dataset Lets You Practice

Category -> functions / transformations (2 of 2)

CATEGORY	FUNCTIONS / TRANSFORMATIONS
Nulls	<code>dropna()</code> , <code>fillna()</code> , <code>isNull()</code>
Duplicates	<code>dropDuplicates()</code> , <code>distinct()</code>
Arrays	<code>size()</code> , <code>explode()</code> , <code>explode_outer()</code> , <code>array_contains()</code> , <code>sort_array()</code>
Structs	Nested field access
JSON	<code>from_json()</code> , <code>to_json()</code>
Nested data	Struct + Array + Array of Struct
Partitions	<code>repartition()</code> , <code>coalesce()</code>
Set operations	<code>union()</code> , <code>unionByName()</code>
Windows	<code>row_number()</code> , <code>rank()</code> , <code>dense_rank()</code>
Conditional logic	<code>when()</code> / <code>otherwise()</code>

84 tasks. 25 levels. One dataset.

`select() -> filter() -> groupBy() -> explode() -> from_json()/to_json() -> window functions -> Kafka streaming.`

Ready for Spark & Big Data interviews.